

Requisitos

1. Diseñar el algoritmo usando divide y vencerás.

tengo una función mayores la cual usa la técnica de divide y vencerás . El arreglo se divide en dos mitades para reducir el problema en partes más pequeñas. Cada mitad obtiene su mayor y segundo mayor, y luego ambos resultados se combinan para encontrar el resultado

mi caso base es

```
if (i == f) {  
    int[] auxiliar = new int[2];  
    auxiliar[0] = array[i];  
    auxiliar[1] = -1;  
    return auxiliar;  
}
```

cuando el subarreglo le queda solo un elemento ese es ,el único maximo de esa sección como no existe ni un elemento con qué compararlo le agregamos un -1

para sacar el medio usa :

```
int medio = (i + f) / 2;
```

Luego se resuelven recursivamente ambas mitades.

2. Explicar cómo se combinan los resultados.

En cada llamada recursiva se devuelve un arreglo con dos posiciones:

- resultado[0]: el mayor del subarreglo
- resultado[1]: el segundo mayor del subarreglo

luego ambas mitades se combinan

se compara el mayor de la izquierda , con el mayor de la derecha , para saber cual es el mayor sub arreglo completo .

el segundo mayor no siempre se encuentra en solo una mitad por eso deben haber varias variables

el segundo mayor de la izquierda (izq[1])

el segundo mayor de la derecha (drc[1])

el mayor de la izquierda (izq[0])

el mayor de la derecha ($drc[0]$)

la combinación , se hace al fusionar los dos resultados de ambas mitades , y al tener a los dos mayores de todo el arreglo .De esta forma, al final de la combinación se obtienen los dos mayores valores del arreglo completo.

3. Obtener $T(n)$.

Se divide en 2 mitades $\rightarrow 2T(n/2)$

Se hacen 2 llamadas recursivas

Se comparan como máximo 4 valores:

$izq[0], izq[1], drc[0], drc[1] = O(1)$

$T(n) = 2T(n/2) + O(1)$

4. Analizar la complejidad temporal Big O.

La complejidad temporal del algoritmo es:

$O(n)$

por que cada elemento del arreglo participa en un número constante de comparaciones en la reducción , y aun así se ven todos los elementos una vez por cada combinación